

Part1

1. Raw input: accX, accY, accZ
2. Feature type: Spectral features. Number of NN classifier input features: 33 features. At least five actual feature names you can include: accX RMS, accX Peak 1 Freq, accX Peak 1 Height, accY RMS, accZ RMS, accY Spectral Power 2.0 - 5.0, accZ Spectral Power 2.0 - 5.0.
3. The NN classifier outputs two class probabilities: nominal and off.
4. The anomaly detector uses spectral features from the accelerometer data. In the selected anomaly feature set, four features are used: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0–5.0, and accZ Spectral Power 2.0–5.0. The K-means anomaly detector outputs one anomaly score, where a higher score indicates that the sample is farther from the normal training clusters.

```
Starting inferencing in 2 seconds...
```

```
Sampling...
```

```
Timing: DSP 44 ms, inference 612 us, anomaly 6 ms, postprocessing 49 us
```

```
#Classification predictions:
```

```
  nominal: 0.003906
```

```
  off: 0.996094
```

5. Anomaly prediction: 1000.000000

```
Starting inferencing in 2 seconds...
```

```
Sampling...
```

```
Timing: DSP 44 ms, inference 593 us, anomaly 529 us, postprocessing 5 ms
```

```
#Classification predictions:
```

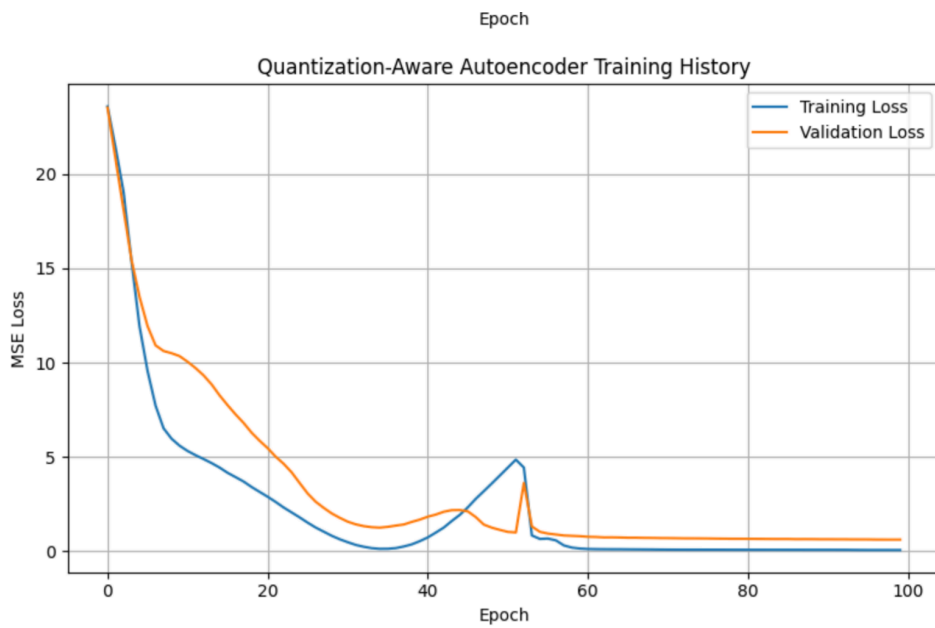
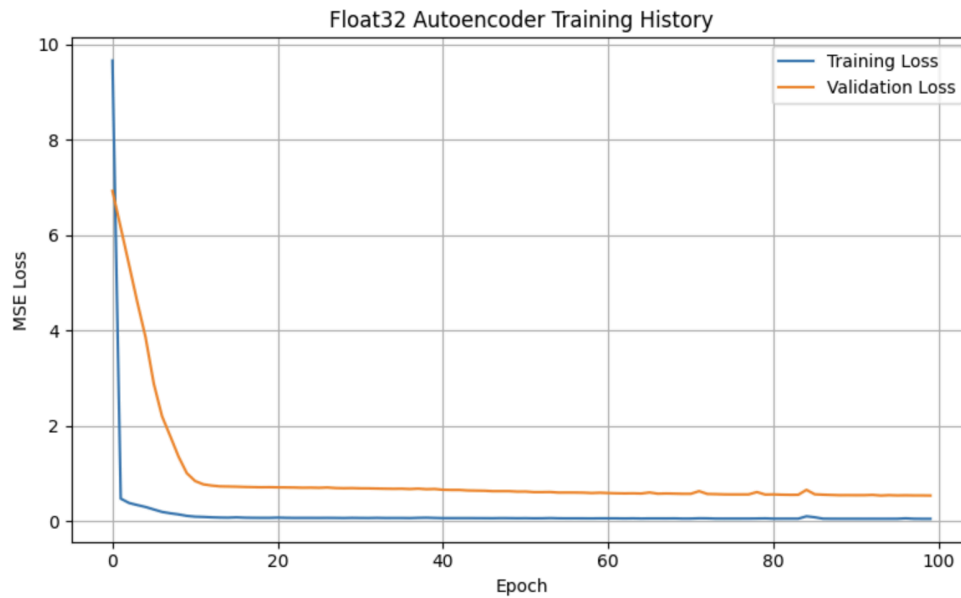
```
  nominal: 0.992187
```

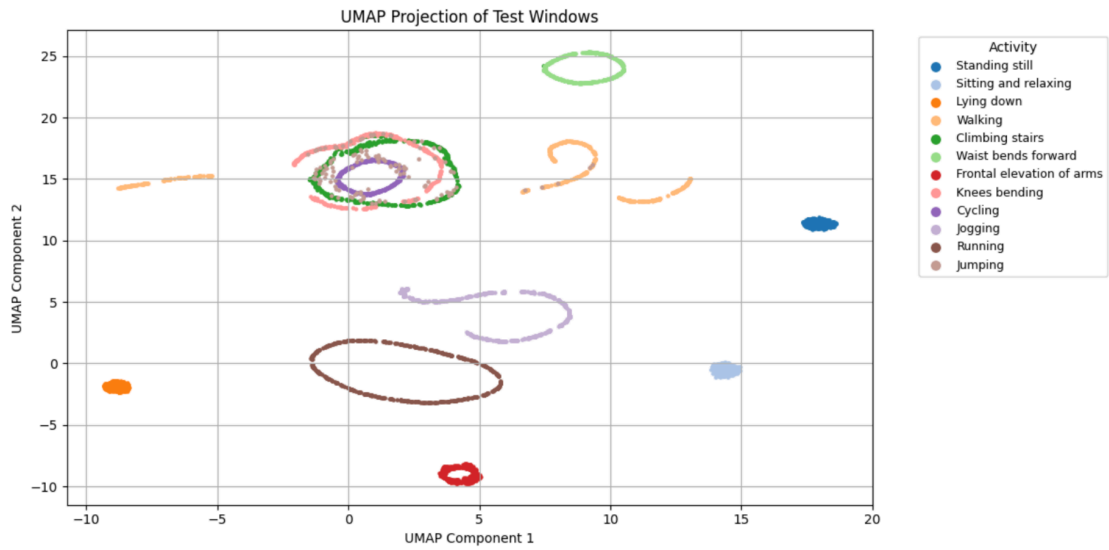
```
  off: 0.007813
```

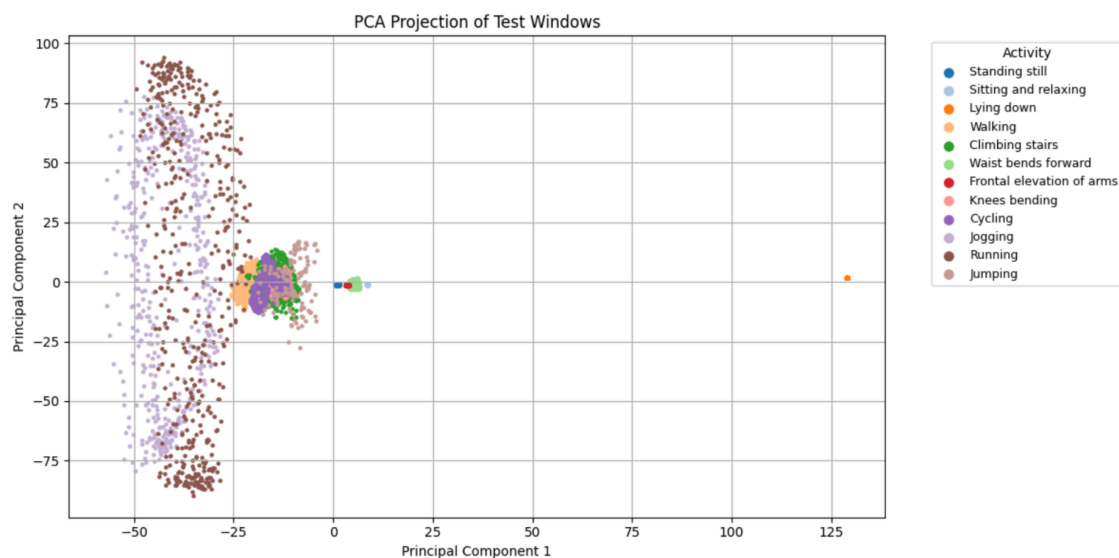
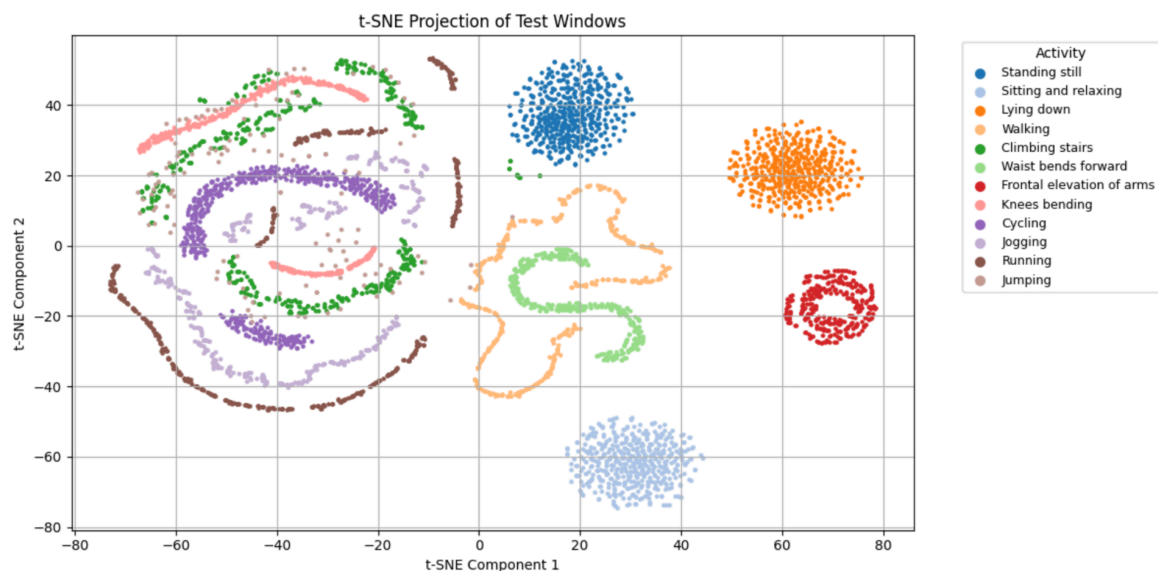
```
Anomaly prediction: 54.285892
```

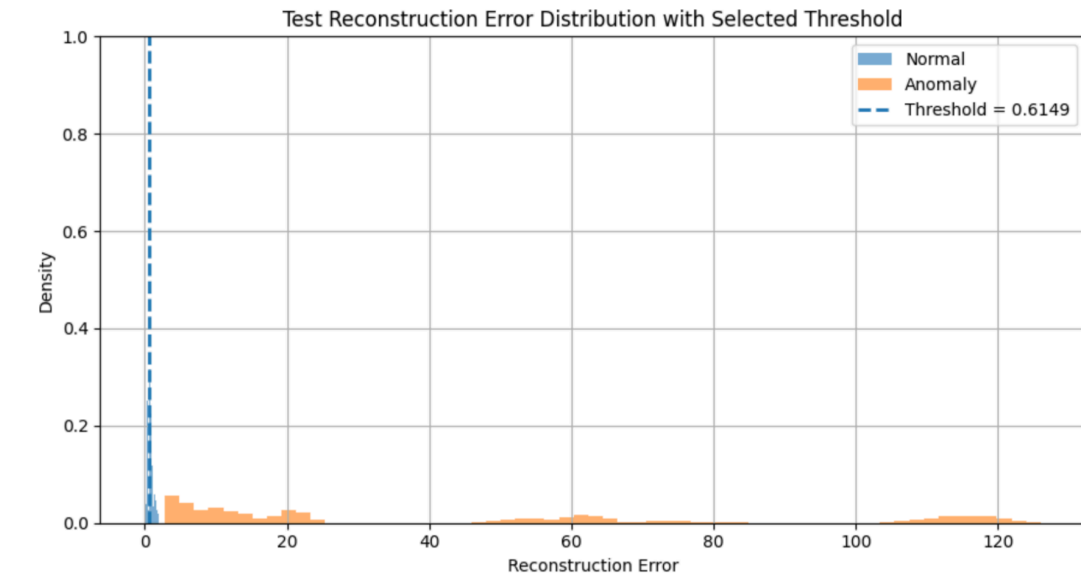
6. The Arduino ran the trained Edge Impulse model locally using accelerometer vibration data. It outputs class probabilities for nominal and off, plus an anomaly score. A low anomaly score means the behavior is similar to training data, while a high score may indicate unusual conditions. The classifier should not always be trusted alone, especially if the sensor placement, fan setup, or vibration conditions differ from training. Therefore, the classifier result should be checked together with the anomaly score.

Part2:



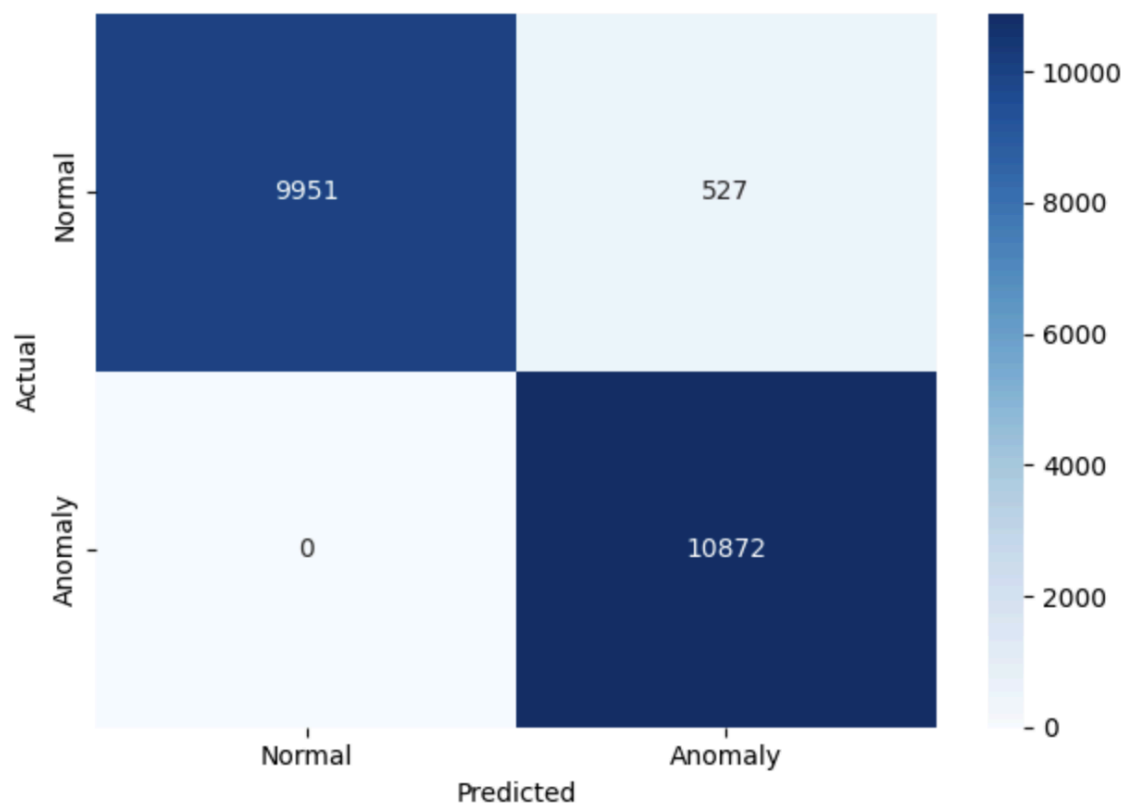




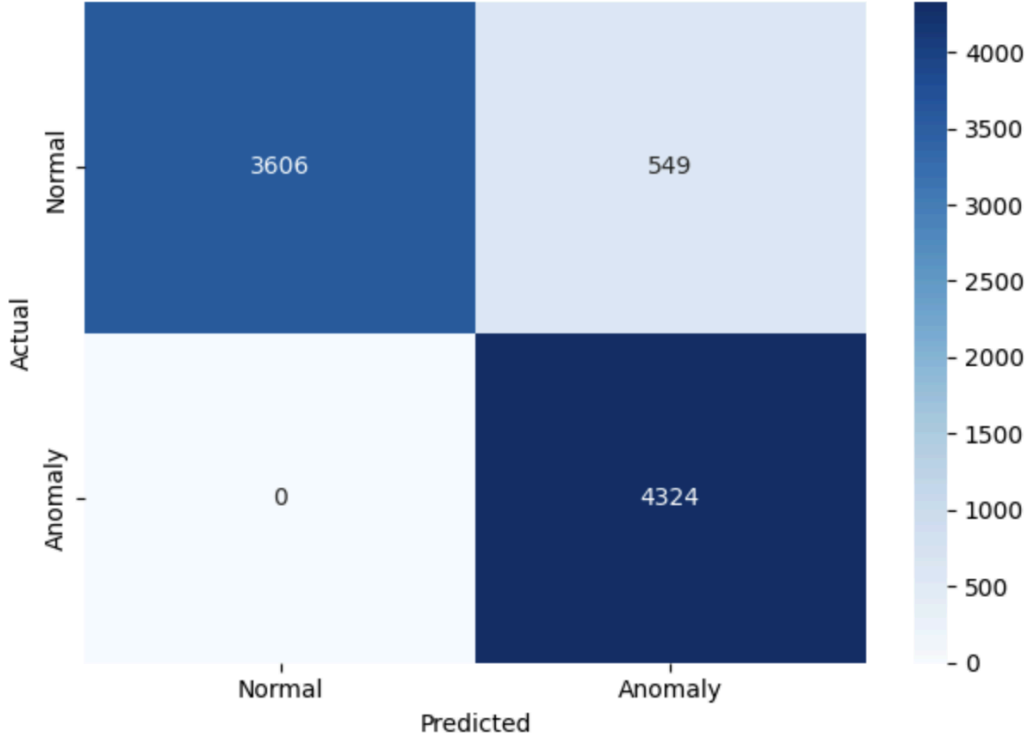


Selected threshold: 0.614929

Train Set Confusion Matrix: Int8 Model



Test Set Confusion Matrix: Int8 Model



Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.87	0.93	4155
Anomaly (1)	0.89	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.94	0.93	0.93	8479
weighted avg	0.94	0.94	0.93	8479

17:27:13.144 -> Result: Anomaly detected
17:27:13.176 -> Reconstruction error: 36.047070
17:27:13.176 -> Result: Anomaly detected
17:27:13.209 -> Reconstruction error: 37.994625
17:27:13.209 -> Result: Anomaly detected
17:27:13.240 -> Reconstruction error: 36.759624
17:27:13.240 -> Result: Anomaly detected
17:27:13.240 -> Reconstruction error: 35.078228
17:27:13.274 -> Result: Anomaly detected
17:27:13.274 -> Reconstruction error: 33.494499
17:27:13.274 -> Result: Anomaly detected
17:27:13.306 -> Reconstruction error: 32.775414
17:27:13.306 -> Result: Anomaly detected

17:27:09.724 -> Result: Anomaly detected
17:27:09.757 -> Reconstruction error: 1.566975
17:27:09.757 -> Result: Anomaly detected
17:27:09.789 -> Reconstruction error: 1.703665
17:27:09.789 -> Result: Anomaly detected
17:27:09.822 -> Reconstruction error: 1.912331
17:27:09.822 -> Result: Anomaly detected
17:27:09.855 -> Reconstruction error: 1.997910
17:27:09.855 -> Result: Anomaly detected
17:27:09.855 -> Reconstruction error: 1.951599
17:27:09.855 -> Result: Anomaly detected
17:27:09.887 -> Reconstruction error: 1.965977

1. I chose a reconstruction-error threshold of 0.614929 for anomaly detection. This threshold was selected from the reconstruction error distribution so that most normal samples stayed below the threshold, while samples with much larger reconstruction error were classified as anomalies. This worked well in my evaluation because the test set achieved about 94% accuracy, with anomaly recall of 1.00, meaning the model detected all anomaly samples in the test set.

2. The autoencoder is trained mainly to reconstruct normal activity windows. When the input is normal, the model can reconstruct it well, so the reconstruction error is low. When the input is an anomalous activity, the pattern is different from what the autoencoder learned, so the reconstructed output is worse and the reconstruction error becomes higher. Therefore, reconstruction error acts like an anomaly score: values below the threshold are treated as normal, and values above the threshold are treated as anomalies.
3. The Arduino deployment must preserve the generated INT8 model bytes in `autoencoder_model.cc`, the selected anomaly threshold, the input window size, the input feature size, and the feature ordering. In this notebook, each input window uses $100 \text{ samples} \times 3 \text{ accelerometer axes} = 300 \text{ input values}$. The Arduino sketch must also use the same type of sensor input and flatten the data in the same order as the notebook. The dataset uses left-ankle accelerometer X, Y, and Z channels as the main features, so the deployed code should match those assumptions as closely as possible.
4. It is important because the model was trained on data with a specific format, window size, feature order, and scale. If the Arduino sketch collects or formats the sensor data differently, the model input will no longer match the training data distribution. This can cause the reconstruction error to become artificially high and may lead to false anomaly detections. For example, using a different accelerometer unit, a different sampling rate, or a different axis order could make normal activity look anomalous to the model.
5. The main limitation is that the model only knows the normal patterns seen during training, so it may flag unfamiliar but harmless movements as anomalies. It is also sensitive to sensor placement, sensor type, sampling rate, and preprocessing differences between the Python notebook and the Arduino. Because the model is quantized to INT8 and runs on a microcontroller, it has limited memory and computation, so the autoencoder must be small. Another limitation is that the threshold is fixed, so it may not work equally well for every person, device position, or environment.